Class 14: Structured Data, Encoding, and Embeddings

Data-driven Systems Engineering

Agenda

- From data type to data structure: structured, semi-structured, unstructured.
- Where each structure shows up in the course's running cases.
- Why raw values must be encoded before a model can use them.
- One-hot, ordinal, target, and cyclic encoding; high cardinality and unseen values.
- Encoding versus embedding: a representation designed a priori versus one learned from data.
- PCA as dimensionality reduction and as a way to learn an embedding; scores, loadings, and latent space.

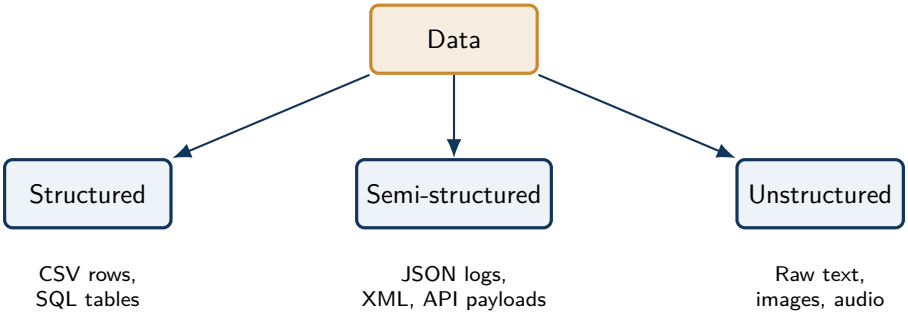
From data type to data structure

- Class 13 asked whether data is tabular, time series, spectral, or image.
- This class asks an orthogonal question: how organized is it, structured, semi-structured, or unstructured?
- Any of the four data types from Class 13 can appear in any of these three organizational forms.
- The organizational form determines how much encoding work is needed before a model can use the data at all.

Structured, semi-structured, and unstructured data

Structured	Fixed schema, fits directly into rows and columns: a CSV file or a SQL table.
Semi-structured	Has organization (keys, tags, nesting) but no fixed column schema: JSON logs, XML documents, API payloads.
Unstructured	No inherent schema at all: raw text, images, and audio waveforms.

The taxonomy as a tree



The three structures in the running cases

Source	Raw form	Structure	Work before modelling
Pasteurization service	JSON events from the `/stream` endpoint of `pasteurization/serving.py`	Semi-structured	Fix the key set, flatten each event into a row, encode `state`
Pasteurization, after flattening	Design matrix, one row $\mathbf{x}_t$ per event	Structured	Encode categorical columns, standardize numerical ones
Fraud detection	Transaction records	Structured	Encode categorical columns, some of high cardinality (e.g. merchant ID)
Production complements	Operator notes, images, logs	Unstructured	Learn or reuse an embedding

The structure is a property of the form in which data arrives, not of the data itself: the same pasteurization measurements are semi-structured on the wire and structured in the design matrix.

Structured text: one record, three text formats

CSV

```
timestamp,state,T,pH
162.0,Hold,71.987,6.663
163.0,Hold,71.990,6.661
```

JSON

```
{"timestamp": 162.0,
 "state": "Hold",
 "T": 71.987,
 "pH": 6.663}
```

XML

```
<event>
  <timestamp>162.0</timestamp>
  <state>Hold</state>
  <T>71.987</T>
  <pH>6.663</pH>
</event>
```

- Structured text is human-readable text whose syntax follows formal rules, so a parser can read it without guessing.
- CSV relies on position (column order and a header), JSON and XML on named keys or tags; the latter allow nesting and optional fields.
- Text format is not the same as structure: the same measurement is structured as CSV and semi-structured as JSON or XML.

Semi-structured example: a streaming JSON event

```
data: {
  "timestamp": 162.0,
  "batch_id": 1,
  "state": "Hold",
  "T": 71.987,
  "pH": 6.663,
  "Kappa": 4.79,
  "Mu": 1.624,
  "Tau": 0.978,
  "Q_in": 0.0,
  "Q_out": 0.0,
  "P": 1.199,
  "dTdt": -0.008
}
```

One event emitted by the `/stream` endpoint of `pasteurization/serving.py` (batch 1, Hold phase); values rounded to three decimals.

- Each event is a self-describing object: every value is paired with the key that names it, so the record carries its own schema.
- The schema is implicit, defined by the producer rather than enforced by the format. The same service returns a differently shaped object, `{"error": "..."}`, from `/point` when a request falls outside the batch range.
- Converting the stream into structured data requires fixing a key set: each event becomes one sample $\mathbf{x}_t$ of the design matrix, with `timestamp` as the index and `state` as a categorical variable.

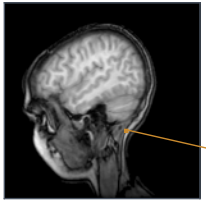
Unstructured data

Text: token sequence of variable length

Batch 1, shift B. Slight foaming observed during filling; holding temperature stable at 72 °C. Outlet valve responding slowly, maintenance requested before the next run.

No fields: quantities, events, and entities must be extracted from the text.

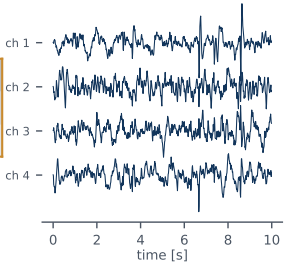
Image: $I \in \{0, \ldots, 255\}^{256 \times 256}$



pixel values

28	89	190
33	141	211
75	190	227

Signal: EEG, 4 channels, $f_s = 80$ Hz



Text: illustrative operator note. Image and signal: magnetic resonance imaging (MRI) slice and electroencephalography (EEG) recording from the Matplotlib sample data.

- No schema: meaning is not attached to named fields and must be extracted (quantities and entities in text, structures in images, events in signals).
- The dimension is large and often variable (document length, image resolution, recording duration), so a learned representation, an embedding, is usually required.
- Neither running case is unstructured, but production fraud and process-monitoring systems often complement their tabular core with free-text notes, images, or logs.

Representations: encoding and embedding

A representation is a map $\phi : \mathcal{V} \to \mathbb{R}^k$ that gives each raw value v a numerical vector; every model operates on $\phi(v)$, never on v. Two families:

	Encoding	**Embedding**
Construction	Fixed a priori by the analyst	Estimated from data by optimizing an objective
Dimension	Set by the scheme (K for one-hot, 1 or 2 otherwise)	Chosen by the analyst, typically $k \ll K$
Geometry	No similarity unless designed	Similar values map to nearby vectors
Typical use	Low-cardinality nominal, ordinal, cyclic	High-cardinality categories, text, images

Neither family compresses by definition: one-hot expands one column into K.

Encodings

An encoding is a map $\phi_{\text{enc}} : \mathcal{V} \to \mathbb{R}^k$ whose form is specified before training. One-hot, ordinal, and cyclic encodings are fully determined by the analyst; target encoding also estimates one statistic per value from the training labels. The choice depends on the cardinality $K = |\mathcal{V}|$ and the measurement scale of the variable.

Encoding	$\phi_{\text{enc}}(v)$	k	Appropriate when
One-hot	$\mathbf{e}_v \in \{0,1\}^K$	K	Nominal scale, small K
Ordinal	$r(v) \in \{1,\dots,K\}$, r order-preserving	1	Ordinal scale: the order is meaningful
Target	$\hat{\mu}_v = \dfrac{n_v \bar{y}_v + m\,\bar{y}}{n_v + m}$	1	Nominal scale, large K; estimated out-of-fold to avoid target leakage
Cyclic	$\big(\sin(2\pi v/P),\ \cos(2\pi v/P)\big)$	2	Periodic variables with period P (e.g. hour of day, $P = 24$)

n_v: number of training samples with value v; $\bar{y}_v$: their mean target; $\bar{y}$: global mean; $m \geq 0$: smoothing weight, which shrinks $\hat{\mu}_v$ towards $\bar{y}$, more strongly when n_v is small.

Why encoding exists

- Models compute with numbers; most raw variables (categories, labels, identifiers) are not numbers.
- Encoding is the deliberate, documented choice of how a symbolic value becomes a numeric one.
- A bad encoding can inject a false sense of order, leak information, or blow up dimensionality.
- The right choice depends on cardinality (how many distinct values) and semantics (does order mean anything).

The Titanic dataset

- Passenger records from the 1912 sinking of the RMS Titanic; the training set has 891 passengers.
- Task: predict `survived` (1 = survived, 0 = did not), a binary classification problem; about 38% survived.
- Used in Class 4 (`Class4_DataEngineering.ipynb`) and here as a small table with every kind of variable that needs encoding.

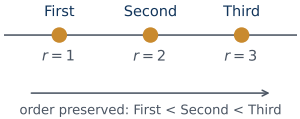
`Column`	**Kind**
`survived`	Target (binary)
`pclass`	Ordinal: 1st, 2nd, 3rd class
`sex`	Nominal, $K = 2$
`embarked`	Nominal, $K = 3$ (S, C, Q)
`age, fare`	Numerical
`sibsp, parch`	Counts (family on board)
`name, ticket, cabin`	Text or identifiers, high cardinality

Encoding examples

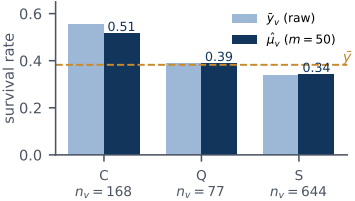
One-hot: embarked

v	C	Q	S
S	0	0	1
C	1	0	0
S	0	0	1
S	0	0	1
S	0	0	1

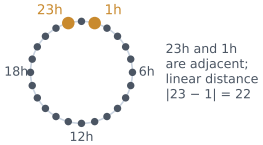
Ordinal: pclass



Target: embarked → survival



Cyclic: hour of day



Titanic data used in `notebooks/Class4_DataEngineering.ipynb` (one-hot: first five samples; target: all samples with known port).

Target encoding: a worked example

1. Category means. Each value v is replaced by the mean target of the training samples with that value (Titanic, $y =$ survived).

Port v	n_v	$\sum y$	$\bar{y}_v$
C	168	93	0.554
Q	77	30	0.390
S	644	217	0.337
All	889	340	$\bar{y} = 0.382$

The nominal column becomes a single numerical column, whatever the cardinality K; one-hot encoding would require K columns.

2. Smoothing. Rare values yield unreliable means; $\hat{\mu}_v$ is a weighted average of $\bar{y}_v$ and $\bar{y}$:

$$\hat{\mu}_v = \frac{n_v \bar{y}_v + m\,\bar{y}}{n_v + m}, \qquad m = 50.$$

- Port C ($n_v = 168$): $\hat{\mu}_C = \frac{93 + 50 \cdot 0.382}{218} = 0.514$.
- Hypothetical port ($n_v = 2$, both survived): $\bar{y}_v = 1.0$, but $\hat{\mu}_v = \frac{2 + 19.1}{52} = 0.41$.

3. Leakage control. If a sample's own y_i enters its encoding, the feature reveals the target. Training samples are therefore encoded out-of-fold; test samples use the means estimated on the training set.

High cardinality and unseen values

Further encodings for large K

Count $\phi(v) = n_v$ (or n_v/n), $k = 1$. Unsupervised, so no target leakage; values with equal counts become indistinguishable.

Hashing $\phi(v) = \mathbf{e}_{h(v) \bmod B} \in \{0,1\}^B$, $k = B$ fixed whatever K. Unseen values need no special handling; distinct values can collide, and the map cannot be inverted, so interpretability is lost.

Target $\hat{\mu}_v$, $k = 1$ (previous slide). Supervised; requires out-of-fold estimation.

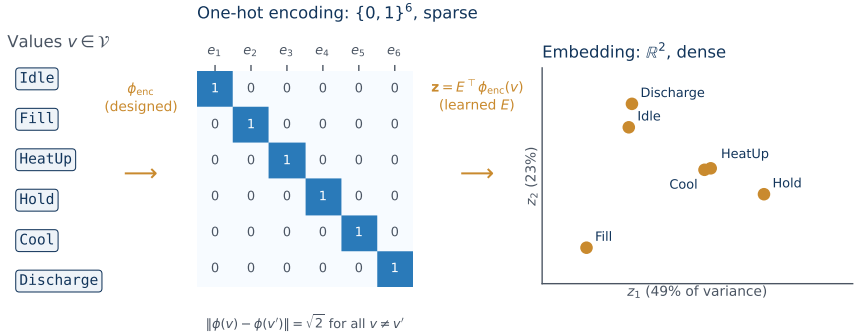
Values not seen in training

- **One-hot:** all-zero vector (`handle_unknown="ignore"`) or a shared "infrequent" column (`min_frequency` in scikit-learn).
- **Target:** with $n_v = 0$ the smoothed formula gives $\hat{\mu}_v = \bar{y}$, so the fallback is built in.
- **Ordinal:** no safe default; the position of a new level must be decided explicitly.
- **Embedding:** a reserved row for unknown values ($\langle$`UNK`$\rangle$), trained on the rarest values.

Rare values should be grouped before fitting: their encodings and embeddings are estimated from few samples.

Embeddings

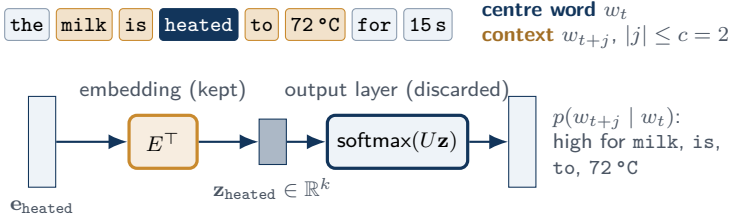
An embedding is a learned map $\mathbf{z} = E^\top \phi_{\text{enc}}(v)$, with $E \in \mathbb{R}^{K \times k}$ and $k \ll K$: each row of E is the dense vector assigned to one value, estimated from data jointly with a model or through a dedicated objective (e.g. word2vec for text).

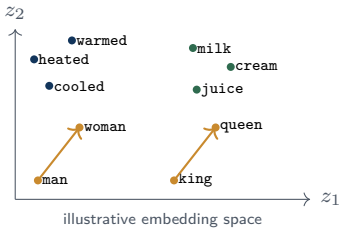


One-hot encoding treats all categories as equidistant; the embedding places phases with similar sensor behaviour close together (HeatUp and Cool; Idle and Discharge). Embedding obtained by principal component analysis (PCA) of the standardized phase-wise means of nine sensor variables over 100 batches simulated with `synth_sensors.py`.

word2vec: embeddings learned from context

word2vec (Mikolov et al., 2013) learns the matrix E from raw text under the *distributional hypothesis*: words that occur in similar contexts have similar meanings.





Training (skip-gram): for every word in the corpus, the model learns to predict the words in its context window. Words used in similar contexts therefore obtain nearby vectors, and some relations become approximately parallel offsets, e.g. $\mathbf{z}_{\text{king}} - \mathbf{z}_{\text{man}} + \mathbf{z}_{\text{woman}} \approx \mathbf{z}_{\text{queen}}$. Such analogies hold only in favourable cases, and part of their success comes from excluding the query words from the nearest-neighbour search (Nissim et al., 2020).

How embeddings are obtained

All routes produce $\mathbf{z} = E^{\top}\phi_{\text{enc}}(v)$; they differ in what makes two values close.

Route	Values are close when	Example
Matrix factorization	Similar statistical profiles	PCA of phase-wise sensor means
Supervised	Similar effect on the target	Entity embedding of a merchant ID
Self-supervised	Similar contexts	word2vec
Pre-trained	Similar in an external corpus	Sentence encoders for operator notes

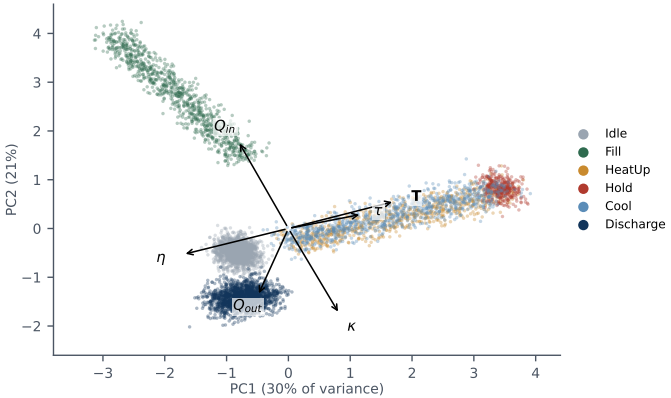
Next: matrix factorization with principal component analysis (PCA).

PCA intuition

PCA rotates the original feature space to new orthogonal axes called principal components, ordered by the amount of variance they explain.

- The first component captures the largest possible variance; later components capture what is left, subject to orthogonality.
- Keeping only the leading components compresses the representation while trying to preserve most of the structure.
- PCA must be fit only on training data, then applied identically at inference time.

PCA projection with feature directions



PCA of 6,362 sensor samples (20 batches simulated with `synth_sensors.py`, nine standardized variables); points are coloured by process phase and arrows show the loading of each sensor on PC1 and PC2. The arrows are rescaled by a common factor for readability: only their directions and relative lengths are meaningful. The phases form distinct regions even though the phase label was not used by PCA.

Scores, loadings, and latent space

For centred (here also standardized) data $X_c \in \mathbb{R}^{n \times p}$, PCA with k components gives

$$T = X_c W_k \quad \text{(encoder)}, \qquad \hat{X}_c = T W_k^\top \quad \text{(decoder)}, \qquad W_k \in \mathbb{R}^{p \times k},\ W_k^\top W_k = I.$$

- **Loadings** W_k: each column is the direction of one component, written in the original variables (the arrows of the biplot). They live in the space of the variables.
- **Scores** T: the coordinates of each sample on the components (the points of the biplot).
- **Latent space**: the k-dimensional space in which the scores live; PC1–PC2 spans a 2D latent space, reported with its explained variance.
- **Embedding**: the score vector of one object, i.e. its point in the latent space.

- The loadings make linear latent axes interpretable (next slides: a temperature axis and a flow axis); neural encoders offer no such direct reading.
- "Loadings" denotes either the unit eigenvectors W or $W\Lambda^{1/2}$ (for standardized data, the correlations between variables and components): state which one is reported.
- Probabilistic PCA, $\mathbf{x} = W\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$, makes $\mathbf{z}$ a latent variable in the statistical sense (Tipping and Bishop, 1999).

Two uses of PCA in this class

Use	Rows of the input	n	A point is	PC1 + PC2
Dimensionality reduction	Sensor samples $\mathbf{x}_t$ (20 batches)	6,362	One observation, in the latent space of the samples	$30\% + 21\%$
Embedding of a categorical variable	Phase-wise means $\mathbf{m}_v$ (100 batches)	6	One value of the variable `state`	$49\% + 23\%$

- Same algorithm, different objects: the first projects observations, the second represents the categories of a variable.
- The percentages are not comparable. In the samples, PCA must also explain the variability within each phase (noise, transients); averaging removes it, so the differences between phases dominate the variance of the means.

Embedding with PCA: a worked example (1/3)

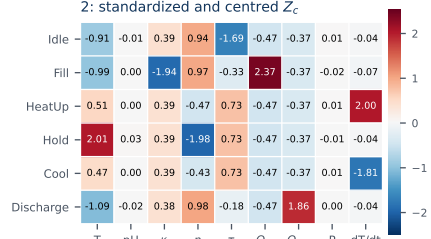
Goal: a $k = 2$ embedding of the $K = 6$ process phases, learned from sensor data (100 batches simulated with `synth_sensors.py`).

1: phase-wise means $M \in \mathbb{R}^{6\times 9}$

	T	pH	κ	η	τ	Q_{in}	Q_{out}	P	dT/dt
Idle	10.4	6.65	4.80	2.19	0.20	0.00	0.00	1.20	0.00
Fill	8.87	6.65	3.90	2.20	0.65	1.50	0.00	1.20	-0.03
HeatUp	40.4	6.65	4.80	1.90	1.00	0.00	0.00	1.20	1.69
Hold	72.0	6.65	4.80	1.60	1.00	0.00	0.00	1.20	0.00
Cool	39.5	6.65	4.80	1.91	1.00	0.00	0.00	1.20	-1.46
Discharge	6.72	6.65	4.80	2.20	0.70	0.00	1.50	1.20	0.00

2: standardized and centred Z_c

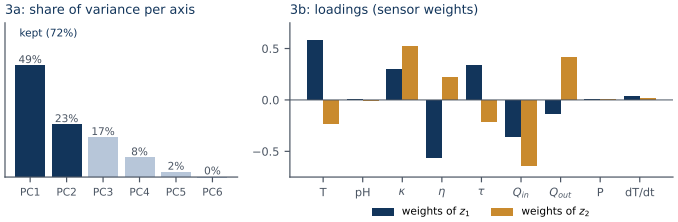
	T	pH	κ	η	τ	Q_{in}	Q_{out}	P	dT/dt
Idle	-0.91	-0.01	0.39	0.94	-1.69	-0.47	-0.37	0.01	-0.04
Fill	-0.99	0.00	-1.94	0.97	-0.33	2.37	-0.37	-0.02	-0.07
HeatUp	0.51	0.00	0.39	-0.47	0.73	-0.47	-0.37	0.01	2.00
Hold	2.01	0.03	0.39	-1.98	0.73	-0.47	-0.37	-0.01	-0.04
Cool	0.47	0.00	0.39	-0.43	0.73	-0.47	-0.37	0.01	-1.81
Discharge	-1.09	-0.02	0.38	0.98	-0.18	-0.47	1.86	0.00	-0.04



- **Step 1:** each phase v is described by the mean of nine sensor variables over its samples (row $\mathbf{m}_v$ of M).
- **Step 2:** each sensor j is standardized, $(m_{vj} - \mu_j)/\sigma_j$, where μ_j and σ_j are the mean and standard deviation of sensor j over all samples, so that variables in different units (°C, mS/cm, cP) become comparable. Because the phases have different durations, the phase means are not centred after this step; each column is therefore centred on the mean of the six phases, so that 0 represents the average phase.

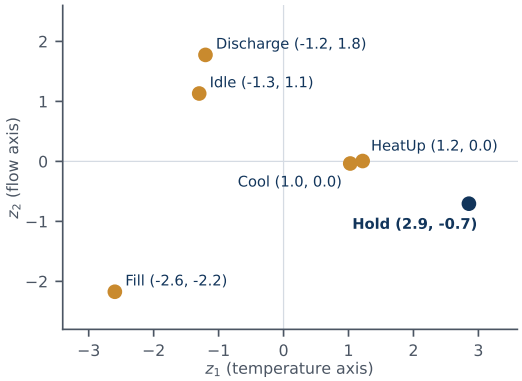
Embedding with PCA: a worked example (2/3)

Step 3: PCA searches for new axes, each a weighted sum of the nine columns of Z_c, along which the phases differ as much as possible.



- **3a:** each bar is the share of the total variation between phases explained by one axis. The first two axes explain $49\% + 23\% = 72\%$, so we keep $k = 2$. PC6 explains 0% by construction: the six centred rows sum to zero, so at most five axes carry variance. With so few points, the shares are fragile estimates.
- **3b:** the bars are the weights (loadings) of each sensor in the new axes. z_1 weights temperature positively ($+0.58$) and viscosity η negatively (-0.56): it is high for hot, thin milk, i.e. a *temperature axis*. z_2 weights inflow negatively and outflow and conductivity positively: it separates filling from discharge, i.e. a *flow axis*. pH and P receive weight ≈ 0: they do not change between phases. Unlike the axes of word2vec, these axes can be read through their loadings.

Embedding with PCA: a worked example (3/3)



Step 4: the coordinates of a phase are the weighted sums of its values in Z_c (step 2, heatmap) with the weights of step 3b. For Hold:

$$z_1 = \underbrace{0.58 \cdot 2.01}_{T} + \underbrace{(-0.56)(-1.98)}_{\eta} + \underbrace{0.34 \cdot 0.73}_{\tau} + \dots$$

$$= 1.16 + 1.11 + 0.25 + \dots = 2.85$$

Repeating this for every phase and for z_2 gives $E = Z_c W_2 \in \mathbb{R}^{6\times 2}$, the matrix E of the embedding definition: row v is $\mathbf{z}_v = E^\top \mathbf{e}_v$.

- Close points mean similar sensor profiles.
- Hold lies far right: the hottest phase.
- Fill is isolated: the only phase with inflow.
- HeatUp and Cool coincide: they differ only in dT/dt, which lies on PC3 (17%) and is discarded with $k = 2$. This is the cost of compression.

Papers and materials

- Abiteboul, S. (1997). Querying semi-structured data. In *Proceedings of the International Conference on Database Theory (ICDT)*, LNCS 1186, 1–18. Springer.
- Zheng, A., Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.
- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations*, 3(1), 27–32.
- Weinberger, K., Dasgupta, A., Langford, J., Smola, A., Attenberg, J. (2009). Feature hashing for large scale multitask learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 1113–1120.
- Jolliffe, I. T. (2002). *Principal Component Analysis*, 2nd ed. Springer.
- Tipping, M. E., Bishop, C. M. (1999). Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B*, 61(3), 611–622.
- scikit-learn User Guide: encoding categorical features; principal component analysis.
- Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111–3119.
- Nissim, M., van Noord, R., van der Goot, R. (2020). Fair is better than sensational: man is to doctor as woman is to doctor. *Computational Linguistics*, 46(2), 487–497.
- Guo, C., Berkhahn, F. (2016). Entity embeddings of categorical variables. arXiv:1604.06737.
- Bengio, Y., Courville, A., Vincent, P. (2013). Representation learning: a review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.

Take-away message

Every model sees a representation, never the raw data:
choose an encoding by design, or learn an embedding from data,
and make that choice explicit.

Thank you for your attention.

Next: Class 15 — feature engineering: cleaning, curation, and selection.